

Core Model Proposal #422: GCAM Bugfix Fall 2025

Product: Global Change Analysis Model (GCAM)

Institution: Joint Global Change Research Institute (JGCRI)

Authors: Pralit Patel, Patrick O'Rourke, Kalyn Dorheim, Steve J. Smith, Matthew Binsted, Kate Calvin

Reviewers: Marshall Wise, Hamza Ahsan, Kalyn Dorheim

Date committed: 12/1/25

IR document number: PNNL-38758

Related sector: Climate, Transportation

Type of development: code, data, queries

Purpose: Collect various bugfixes and minor improvements that have been floating in various branches and get them into the Core GCAM.

Description of Changes

Remove MAGICC

GCAM contained an older version of MAGICC (v5.3). Hector is used as the default model and we have stopped maintaining the ability to run MAGICC. In this proposal, the MAGICC C++ code (including references to MAGICC in `world.cpp`) and input files are removed.

List of files affected:

- `climate/*`
- `containers/include/imodel_feedback_calc.h`
- `containers/ world.h/cpp`
- `containers/source/scenario.cpp`
- `emissions/source/luc_emissions_summer.cpp`
- `reporting/source/xml_db_outputter.cpp`
- `util/base/include/gcam_data_containers.h`
- `input/gcamdata/data-raw/generate_package_data.R`
- `input/gcamata/inst/extdata/mi_headers/ModelInterface_headers.txt`
- `Input/gcamdata/R/constants.R`
- `input/gcamdata/R/zclimate_xml_magicc.R`
- `input/gcamdata/R/zclimate_xml_no_climate_model.R`
- `input/gcamdata/R/zmodeltime_L200.modeltime.R`
- `input/magicc/*`

Remove MAGICC Specific Behaviors

We run the climate model after each model time step has finished solving. As well as a full re-run of the entire century at the very end of the simulation. In reality, only Hector was able to run by time period and the later was in place for MAGICC. While it did not influence results to run Hector again at the end of the simulation it certainly confused users checking detailed logs or results from Hector. Given this and MAGICC's removal we no longer re-run the climate model as the end of the module simulation.

We remove generating certain emissions species such as HFC245fa which were specific to MAGICC.

List of files affected:

- `containers/source/world.cpp`

Correct Reporting issues of some Halocarbon Forcings

There were a couple of issues which lead to improper reporting of Hector Halocarbon forcing results to the GCAM XMLDB. Including gathering un-adjusted forcing results as well as not having updated certain forcing aggregations such as Kyoto or Montreal to query for Hector specific Halocarbon species. In this proposal we now gather the adjusted Halocarbon forcing results and update the XMLDB reporting of forcing aggregations to correctly gather the correct species from a Hector Climate Model.

List of files affected:

- climate/source/hector_model.cpp
- reporting/source/xmldb_outputter.cpp

Make gcam-hector-outputstream a "standard GCAM file"

The "gcam-hector-outputstream" is the main output file for Hector and can be valuable for users who are interested in detailed results, given only select climate model results are available through the XMLDB. The writing of that output file had been hard coded in the HectorModel C++ when coupled to GCAM. Including the output file path, being able to enable / disable output, and being able to differentiate output names by scenario. All of this functionality could be provided automatically by the AutoOutputFile class and can then be set by the user in the configuration.xml / File section. Thus we simply switch from directly outputting the output stream in HectorModel to using AutoOutputFile instead.

List of files affected:

- climate/ hector_model.h/cpp
- exe/Configuration_*.xml
- output/.gitignore

Reduce looping in GCAM Fusion queries for exact matching predicates

GCAM fusion queries need to loop over and access heterogeneous data types which do not necessarily conform to a common hierarchy. Thus the "query" processing tends to be quite naive in nature. In general most queries perform fast enough that it isn't too much of a concern. There are select queries such as getting or setting data into the very long list of markets can bog down. Here we add a bit more infrastructure to the query predicates so that the search algorithm can know when we are doing exact value filters and thus provide an early return opportunity instead of having to loop over thousands of unneeded containers, for instance. This can provide ~ 10% speed up in some cases.

List of files affected:

- util/base/include/gcam_fusion.hpp

Move National Accounts Enum data to DEFINE_DATA

The NationalAccount::mAccounts is a vector of numerous individual "accounts" useful for Macro operation or reporting. Given the relatively large number of accounts it was convenient to store them in a std::vector where the index is an Enum NationalAccount::AccountType. In such a way bulk processing is easier, for instance. The mAccounts member variable was excluded from the standard DEFINE_DATA methodology for fear GCAMFusion wouldn't be able to properly process it. However, that in fact is not the case. And further with a few minor tweaks we can

have gcamwrapper query using Enum values such as AccountType resulting in a clean way to get / set NationalAccount data including GDP.

List of files affected:

- containers/ national_account.h/cpp

Respect Zero Share weight to disable investing in new Resource extraction

The Subresource object was not checking Technology share-weight at all when doing "new investment" in resources. For the most part this is expected as there is not actually any logit competition in the resources at present. However, it would be helpful to use zero share-weight to disable Technology availability as we do it the rest of GCAM. Making the C++ changes is straightforward, however there was some flawed logic for the processing of Geothermal share weights (not noticed previously as share weights had been ignored) which also gets corrected as part of this fix.

List of files affected:

- resource/source/subresource.cpp
- input/gcamdata/R/zenergy_L210.resources.R

Ensure we reset Scenario name prior to XML Parsing

We had reset the Scenario name to be to the value set in the configuration.xml scenarioName String. We did so after parsing all XML files, the reason being because a Scenario name could be potentially set via XMLParse. However in practice we would not want to do that. That being said, in case an AutoOutputFile is opened during XML Parse we needed to reset it earlier. Thus to cover both cases we now reset the Scenario name both before and after XML Parse.

List of files affected:

- containers/source/scenario.cpp

Fix "off by one" error in driver_drake(stop_before=)

The logic of which gcamdata chunks need to be run when setting the option "stop_before" was off by one. I.e. it would attempt to run the chunk which we wanted to stop before. (Note, this feature does work as intended in driver). The fix is a simple adjustment to the chunk run list to exclude the stop_before chunks using the same logic as in driver.

List of files affected:

- input/gcamdata/R/driver.R

Allow users to set future Final Energy Service values.

For EnergyFinalDemand users can set a "base-service" value in the historical periods. The model will then endogenously determine future service levels based off of growth from the "base-service" in the final calibration year using price and income elasticities. There were some limitations to this, and in this update users can now also set "base-service" values in any future period as well. This allows flexibility to do two things. First, a user can now model a final service that begins (or restarts) in the future (e.g. was zero in the final calibration year), which was not possible before. Second, this also lets a user set a pathway for base-service values for one or more future years. For years where the user did not set a base-service value, services will grow from the service values in previous years using elasticity values as before. Thus this adds the ability for a user to more easily fine tune future end-use base service pathways.

List of files affected:

- sectors/source/energy_final_demand.cpp

Have stub-technology use the standard XML Parse

The StubTechnology would directly call the resolved technology instance's XMLParse method to handle any parsed "XML adjustments" which were set by the user. This short cut works fine in most cases however would not work if the "XML adjustments" were changing parameters in the TechnologyContainer itself. Most notably if the user wanted to adjust the initial or final technology available year. In this proposal we instead construct a temporary XML element which contains the "XML adjustment" nodes and use the standard XMLParseHelper / ParseChildData methods to do the parsing ensuring all parsing is consistently handled.

List of affected files:

- technologies/source/stub_technology.cpp
- technologies/source/technology_container.cpp

Add a verbose-parse-warnings option to get more context of XML Parse issues

This is reviving a feature which existed in the old XML Parse algorithm, although we let users configure if they want verbose messages or not via verbose-parse-warnings in the Booleans section. As otherwise it could produce a lot of log messages. In the current Core whenever we generate a warning or parsing error we only print information about the offending XML tags but not any context of where in the XML hierarchy the failure occurs. We can now get a "stack trace" such as follows to get that context:

```
Deleting node: subsector
  From node subsector[name="SugarCropC4_WislaR"]
  From node supplysector[name="SugarCrop"]
  From node region[name="Ukraine"]
  From node world[]
  From node scenario[]
```

In addition, XML parsing of StubTechnology are "deferred" until completeInit to ensure we have a complete set of definitions before making "XML adjustments". If any parsing warning / errors were to occur we would lose parsing context. We partially address that here as well. It would be ideal if we could track the originating XML from which the issue stemmed from, however that provided more involved thus is not included here.

```
XML parsing complete.
Starting new scenario: Reference
XML Readin Time: 10.0277 seconds.
Deleting node: minicam-energy-input
  From node minicam-energy-input[name="biomass"]
  From node period[year="1975"]
  From node [region="USA", sector="regional biomass", subsector="regional
biomass", tech="regional biomass"]
```

Finally note: this has very minimal performance impact on XML parsing. For the sake of reducing log messages we still default to verbose messages being off.

List of files affected:

- technologies/source/stub_technology.cpp
- util/base/source/xml_parse_helper.cpp

Fix Bisection solution timer bug

GCAM keeps timers for various tasks to evaluate performance and identify areas for improvement including the various Solution Components. In the BisectionSolutionComponent there is an early exit condition when all markets are solved in bracketing and therefore no need to bisect. However we had forgotten to stop the Bisection timer in such a case leading occasionally to over-estimating the time spent in that solution component. Here we ensure the timer is correctly stopped in all conditions.

List of files affected:

- solution/solvers/source/bisect_all.cpp

Allow users to configure the output buffer size when writing XMLDB

Writing the XMLDB output from a GCAM scenario takes a significant amount of time, more than 5 minutes locally and even more on HPC systems. There are not many levers we can use to attempt to speed this up, however one which we do control does indeed have significant impact: the "buffer" size for passing data along the GCAM → Apply Output Filters → Store results in an XMLDB chain. In particular using a larger size *seems* to speed up writing by as much as 50%. However, given there are many buffers needed to facilitate the output chain, it implies noticeably higher peak memory usage. Therefore, we allow users to set the size via configuration.xml / Ints section but default to 100 MB which seems to be an optimal for write time speed up. Note the times listed below are in seconds.

buffer (MB)	Mac	Deception	Windows
1	224.286	502.078	651.777
10	216.123	450.707	593.884
50	182.367	236.172	495.535
100	104.22	247.015	647.945
1000	106.039	234.568	

List of files affected:

- java/source/XMLDBDriver.java
- reporting/ xml_db_outputter.h/cpp
- exe/XMLDBDriver.jar

Some updates to the testing framework to update module versions

We pull in changes in the gcam testing frame work to update various module load versions to be up to date with the latest available on the system. In addition we make more robust the check in config.system to determine how to set the JAVA_LIB path which differs between Oracle JDK and OpenJDK installations.

List of files affected:

- build/linux/configure.system
- util/testing-framework

Remove unit conversions from the Transportation Module

Various unit conversions were hard coded into the C++ for the transportation module (TrnTechnology and TrnSubsector) including those such as 1975\$ to 1990\$, Joules to BTUs, and billion to million. These conversions were not always intuitive to users and required them to have to mimic them in gcamdata which was not always straightforward. In this CMP we remove unit conversions from the C++. The natural units of inputs are EJ for quantities and 1975\$/GJ for prices. We have flexibility on what the output units may be. Ideally we keep *numbers* close to one as much as possible, and again avoid having to bake into the C++ unit conversions. From that perspective we decided to change the output quantity units as well. In summary: Previously price units were 1990\$/pass-km and 1990\$/ton-km; quantity units of million-pass-km and million-ton-km. The new units will be **1975\$/pass-km** and **1975\$/ton-km**; quality units of **billion-pass-km** **billion-ton-km**.

List of files affected:

- sectors/source/tran_subsector.cpp
- technologies/ tran_technology.h/cpp
- input/gcamdata/inst/extdata/energy/A25.globaltech_coef.csv
- Input/gcamdata/inst/extdata/energy/A54.sector.csv
- input/gcamdata/R/zenergy_L254.transportation_UCD.R
- input/gcamdata/R/zgcamusa_L254.transportation.R
- output/queries/Main_queries.xml

Update cost conversions within C++ code

The C++ code had a number of cost conversions (from 1975\$ to 1990\$) that were no longer consistent with the `gdp_deflator()` function within GCAM's data system. These have been made consistent within this core model proposal. In addition we take this opportunity to ensure that all code files are using the function utils to get the conversion and, as a slight performance tweak, avoid a function call to get the value by inlining the function util methods.

List of files affected:

- emissions/source/co2_emissions.cpp
- emissions/source/nonco2_emissions.cpp
- functions/source/building_service_function.cpp
- functions/source/ctax_input.cpp
- functions/ function_utils.h/cpp
- land_allocator/source/carbon_land_leaf.cpp
- land_allocator/source/land_leaf.cpp

Code clean up missed in previous CMPs

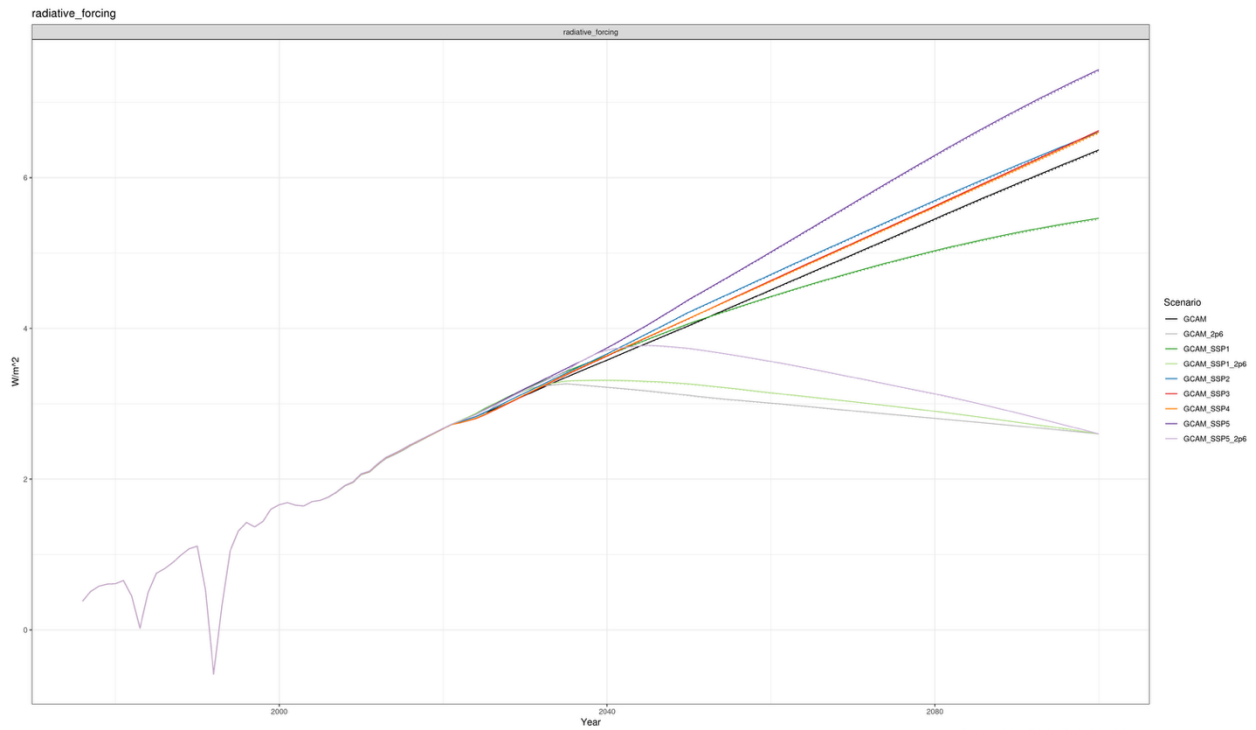
In various prior Core Model Proposals we have cleaned up unnecessary code such as `getNonEnergyCost` or change function signatures from using `std::string` to `gcamstr`. We identified a couple of places which were missed in the previous effort and clean that up in this bug fix. It happens to be the case that these bugs did not affect model behavior, however incorrect function signatures could possibly lead to incorrect behavior.

List of files affected:

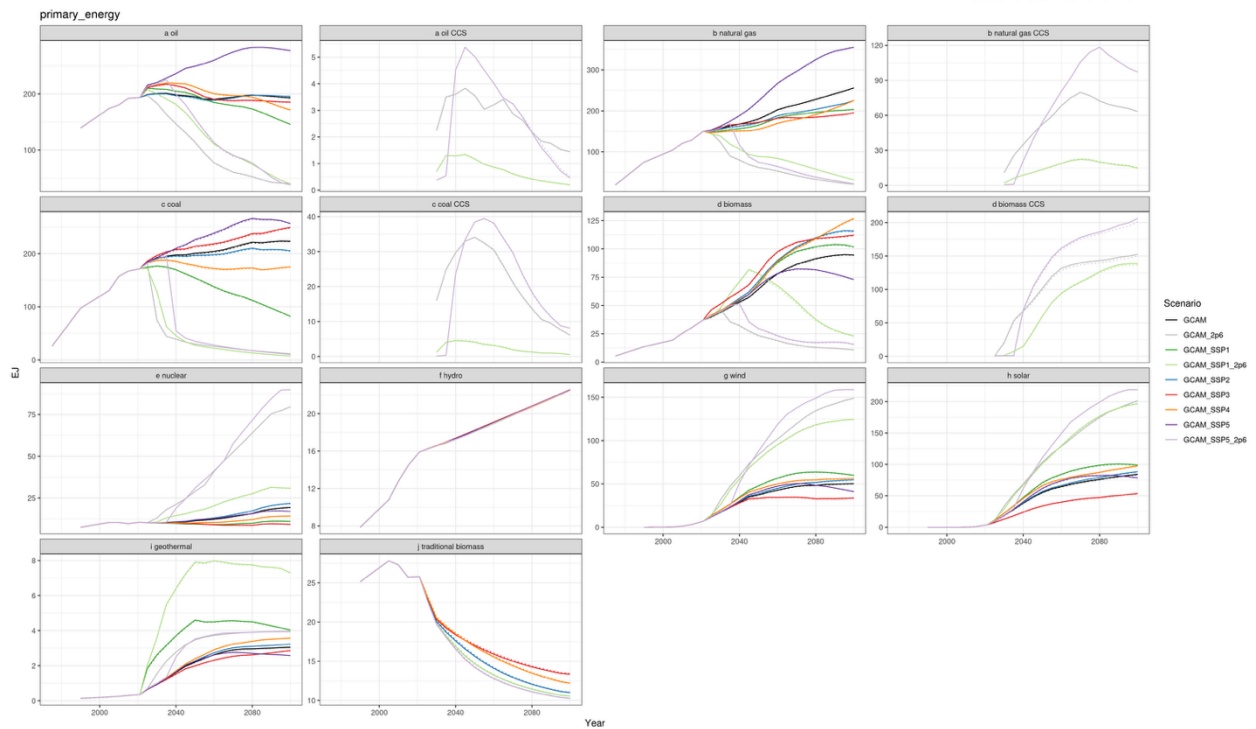
- technologies/ ag_production_technology.h/cpp
- technologies/ nuke_fuel_technology.h/cpp

Validation

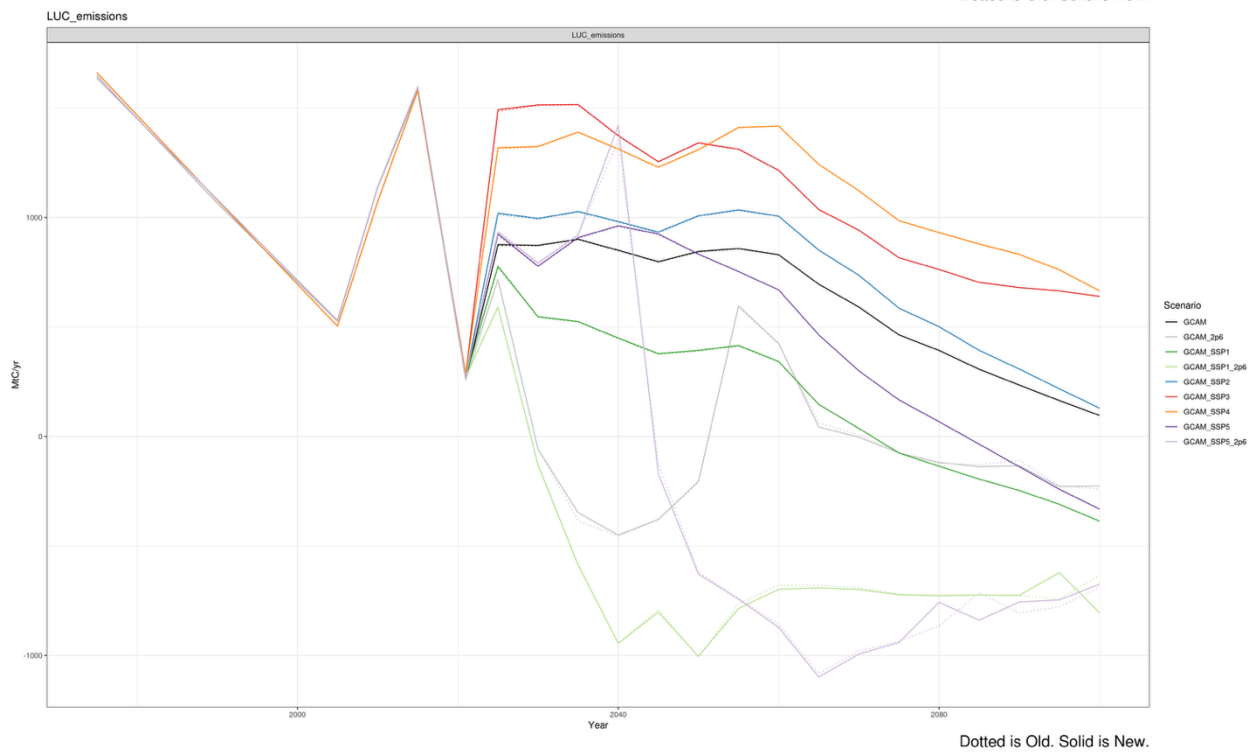
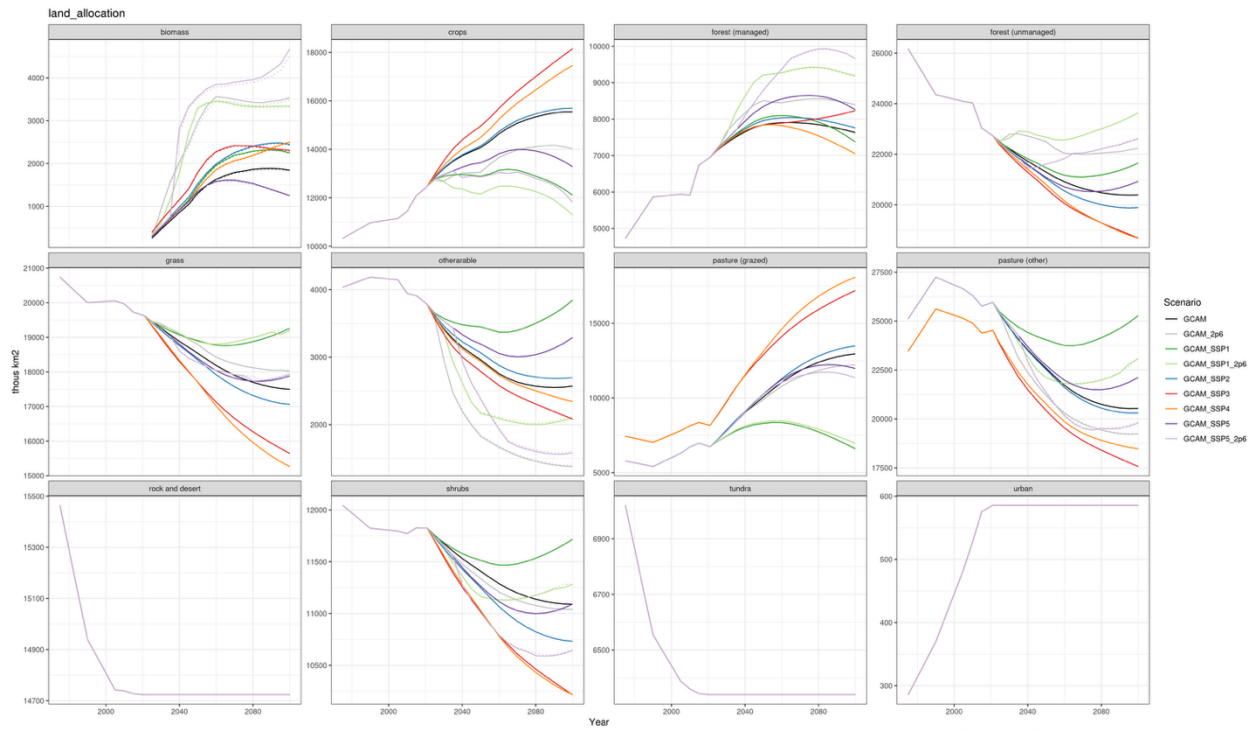
In principle no results should change due to the proposal. However, given the dollar year unit conversions and round off changes from the Transportation units change we do in fact expect minor differences in results.

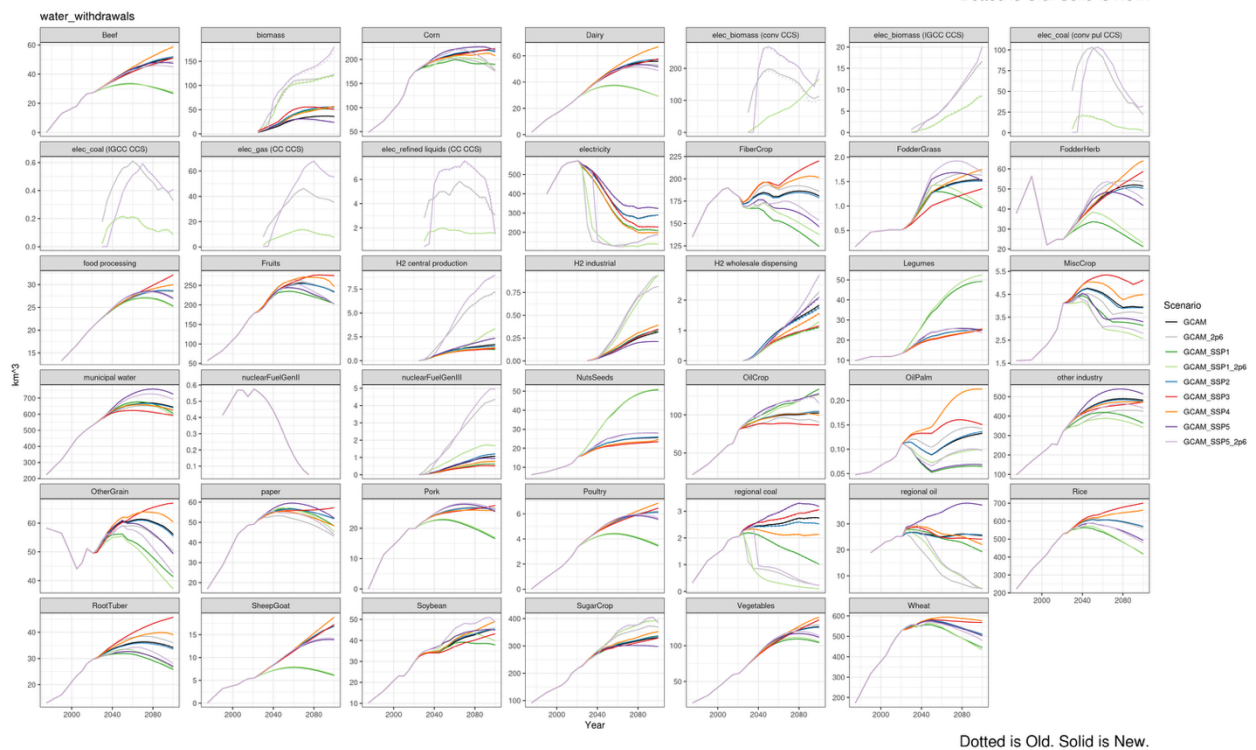
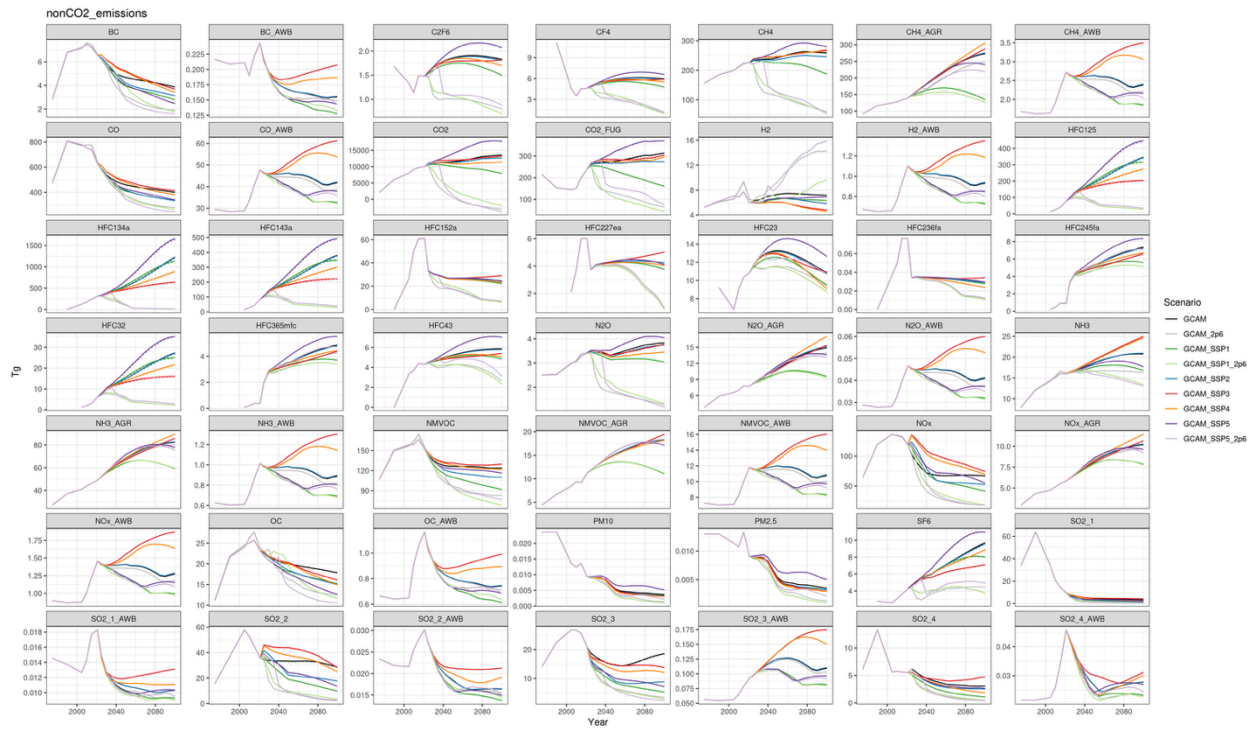


Dotted is Old. Solid is New.

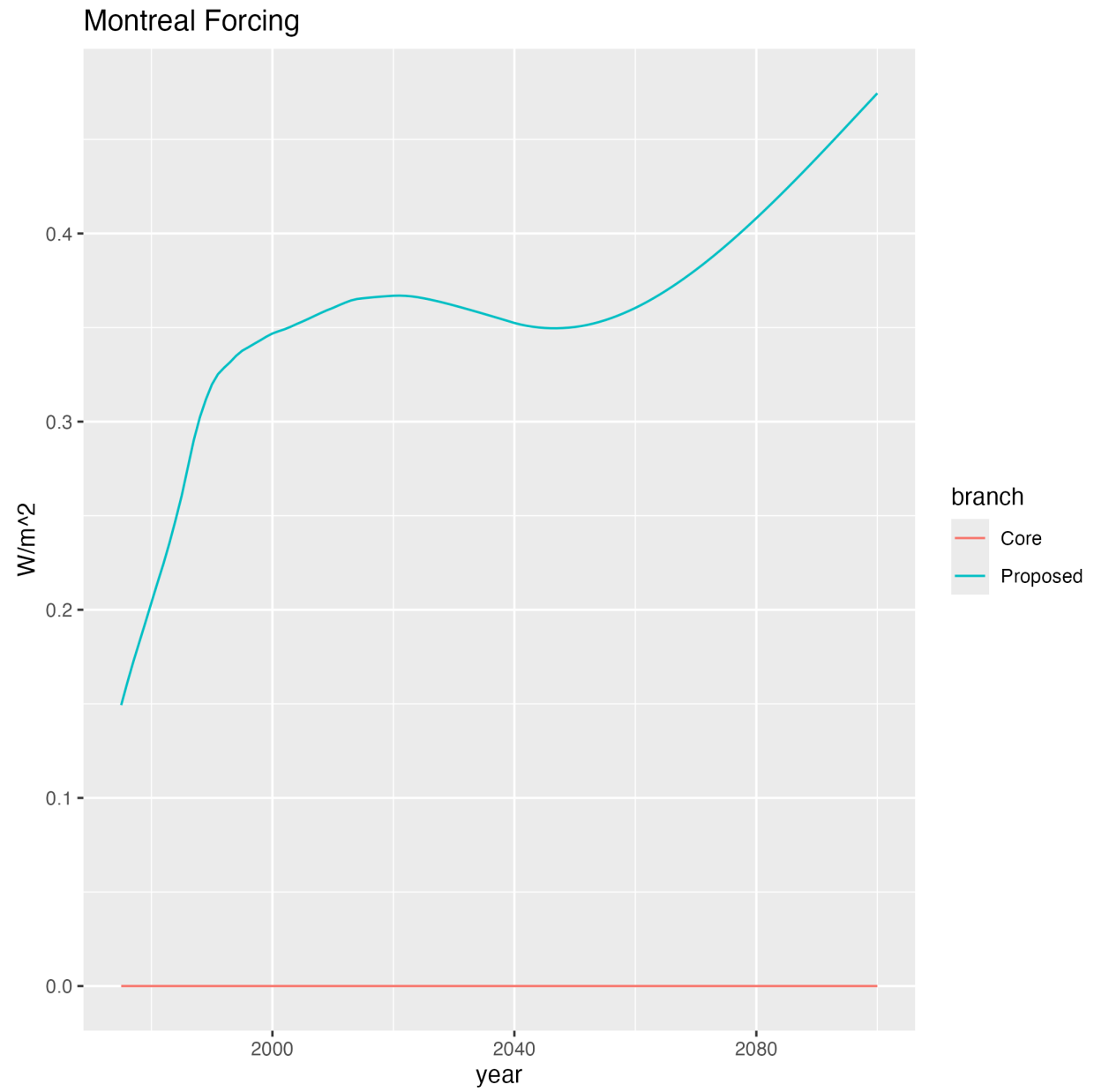


Dotted is Old. Solid is New.





In addition we can verify that the XMLDB results correctly report forcing aggregations such as Montreal Forcing and the Halocarbon forcing are the adjusted values and match those being reported in the main Hector output stream:



CF4 Forcing Normalization Reporting Fix

